Constraint Satisfaction Problems Study Notes

Part 5: Forward Checking and Constraint Propagation

Learning Goal

By the end of this part, you should understand how CSP solvers can detect failure earlier than plain backtracking by actively reducing domains during search. You should be able to explain forward checking, understand the idea of constraint propagation, and grasp the basic meaning of arc consistency and the AC-3 algorithm.

1. Core Idea

Basic backtracking checks whether the current assignment is consistent, but it does not always look ahead far enough. Sometimes a branch is already doomed even though no immediate constraint has been violated yet.

Forward checking and constraint propagation improve search by asking:

- What effect does the current assignment have on the remaining unassigned variables?
- Can we remove values now that are no longer possible?
- Can we detect inevitable failure before going deeper into search?

These methods reduce wasted search by cutting off impossible branches earlier.

> **Things to Note:**
>
> - Backtracking only reacts to visible inconsistency.
> - Forward checking and propagation try to predict future inconsistency earlier.
> - The goal is not to change the solution, but to shrink the search space sooner.

2. Why Basic Backtracking Is Not Enough

Imagine that after a few assignments, an unassigned variable has no legal values left. The current branch is already impossible. But plain backtracking may not notice this until it eventually reaches that variable later in the search.

This delay wastes time.

So we want methods that immediately inspect the effects of new assignments on neighboring variables and reduce their domains when necessary.

> **Main Problem:**

A branch can be dead long before plain backtracking realizes it. Look-ahead methods exist to discover that earlier.

3. Forward Checking

Forward checking is the simplest look-ahead method. Whenever a variable is assigned a value, forward checking looks at all neighboring unassigned variables and removes any domain values that are no longer consistent with the new assignment.

Basic logic

1. Assign a value to a variable.
2. For every unassigned variable connected by a constraint, examine its domain.
3. Remove values that are no longer allowed.
4. If any domain becomes empty, report failure immediately.

So forward checking is a kind of local domain pruning after each assignment.

What it does well

- It catches some dead ends earlier than plain backtracking.
- It is simple and efficient compared with stronger propagation methods.

What it does not do

It only looks one step ahead. It does not fully reason through chains of consequences among unassigned variables.

> **Common Confusion:**
>
> Forward checking is not the same as full propagation. It checks the immediate effect of an assignment on neighboring domains, but it does not necessarily continue reasoning through all indirect consequences.

4. Example of Forward Checking: Map Coloring

Suppose we assign:

```
WA = red
```

If `WA` is connected to `NT` and `SA`, then forward checking removes `red` from the domains of `NT` and `SA`.

So if originally:

- `NT: {red, green, blue}`
- `SA: {red, green, blue}`

after assigning `WA = red`, they become:

- `NT: {green, blue}`
- `SA: {green, blue}`

If one of these domains had contained only `red`, then it would become empty and failure would be detected immediately.

5. Example of Forward Checking: Exam Scheduling

Now use the exam scheduling example with domain:

`{Mon-AM, Mon-PM, Tue-AM}`

Suppose we assign:

`CS101 = Mon-AM`

Given the constraints:

- `CS101 ≠ CS102`
- `CS101 ≠ MATH201`

forward checking removes `Mon-AM` from the domains of `CS102` and `MATH201`.

Variable	Domain Before	Domain After Assigning `CS101 = Mon-AM`
`CS102`	`{Mon-AM, Mon-PM, Tue-AM}`	`{Mon-PM, Tue-AM}`
`MATH201`	`{Mon-AM, Mon-PM, Tue-AM}`	`{Mon-PM, Tue-AM}`
`ENG101`	`{Mon-PM, Tue-AM}` if unary rule already applied	No change unless directly constrained by `CS101`

If, for example, `CS102` had already been restricted to only `Mon-AM`, then this assignment would immediately make its domain empty and the branch would fail at once.

> **High-Yield Point:**
>
> Forward checking performs domain pruning immediately after each assignment. Its main purpose is early failure detection.

6. The Limitation of Forward Checking

Forward checking looks only from the newly assigned variable outward. It does not fully reason about interactions among unassigned variables themselves.

This means some hidden problems can remain undetected.

Example intuition

Suppose assigning one variable reduces a neighbor's domain to a single value. That new restriction may create trouble for another variable connected to that neighbor. Forward checking may not continue this chain deeply enough.

So although it is better than plain backtracking, it is still incomplete as a propagation method.

> **Simple Summary:**
>
> Forward checking sees immediate damage. Constraint propagation tries to follow damage further through the network.

7. Constraint Propagation

Constraint propagation is the broader idea of repeatedly using constraints to reduce domains as much as possible.

Instead of stopping after one local update, propagation keeps pushing the implications of domain reductions through the constraint network.

The main goal is again the same:

- remove impossible values,
- detect failure earlier,
- make search more efficient.

Why propagation is stronger

If one domain shrinks, that may force shrinkage elsewhere. If another domain then shrinks, the effect may continue. Constraint propagation keeps applying these implications until no further reduction is possible.

8. Arc Consistency

One of the most important propagation concepts in binary CSPs is **arc consistency**.

An arc from variable `X` to variable `Y` is arc-consistent if every value in the domain of `X` has some compatible value in the domain of `Y` that satisfies the constraint between them.

Intuition

A value for `X` should not remain in the domain if it has no possible supporting value in `Y`.

If no support exists, that value should be removed.

Simple example

Suppose the constraint is `X ≠ Y`.

If:

- `Domain(X) = {red, green}`
- `Domain(Y) = {red}`

Then the value `red` in `X` has no supporting value in `Y`, because `Y` only has `red` and the constraint requires inequality.

So `red` must be removed from `Domain(X)`, leaving:

```
Domain(X) = {green}
```

> **Things to Think About:**
>
> - Why does a value become useless if it has no support in a neighboring domain?
> - Why is arc consistency especially natural for binary CSPs?
> - Can a CSP be arc-consistent and still have no complete solution?

9. Example of Arc Consistency: Map Coloring

Consider the map-coloring constraint `WA ≠ NT`.

Suppose:

- `Domain(WA) = {red, green}`
- `Domain(NT) = {red}`

Check the arc from `WA` to `NT`.

- The value `WA = red` has no support in `NT`, because `NT` only has `red` and the rule requires different colors.
- The value `WA = green` does have support, because `NT = red` satisfies the inequality constraint.

So arc consistency removes `red` from `WA`, leaving:

```
Domain(WA) = {green}
```

Now imagine `WA` is also connected to `SA`. If shrinking `WA`'s domain affects support for values in `SA`, then those related arcs must also be reconsidered. This is exactly the kind of chain effect that simple forward checking does not fully handle.

10. Example of Arc Consistency: Exam Scheduling

Now use the exam scheduling example with the binary constraint:

```
CS101 ≠ CS102
```

Suppose:

- `Domain(CS101) = {Mon-AM, Tue-AM}`
- `Domain(CS102) = {Mon-AM}`

Check the arc from `CS101` to `CS102`.

- The value `CS101 = Mon-AM` has no support in `CS102`, because `CS102` can only be `Mon-AM` and the constraint requires different slots.
- The value `CS101 = Tue-AM` does have support, because `CS102 = Mon-AM` satisfies the inequality constraint.

Therefore arc consistency removes `Mon-AM` from `CS101`, giving:

```
Domain(CS101) = {Tue-AM}
```

Now suppose `CS101` is also constrained with `MATH201` through `CS101 ≠ MATH201`. If the shrinkage of `CS101` causes some value in `MATH201` to lose support, that arc must also be checked. This is how propagation spreads through the CSP.

> **Things to Note:**
>
> - Arc consistency works on domain support, not on complete assignments.
> - A value is removed when no compatible value exists in the neighboring domain.
> - This is stronger than forward checking because consequences can continue through multiple arcs.

11. Why Arc Consistency Matters

Arc consistency strengthens the search process because it removes values that are already impossible even before they are explicitly tried in backtracking.

This can drastically shrink domains and reveal contradictions early.

However, arc consistency is still not a full solution method for all CSPs. A problem may be arc-consistent and still require search to find a complete assignment.

> **Common Confusion:**
>
> Arc consistency does not guarantee that the CSP is solved. It only guarantees that each remaining value has local support across each arc. Global contradictions may still remain.

12. AC-3 Algorithm

The most commonly discussed algorithm for enforcing arc consistency is **AC-3**.

Its purpose is to repeatedly inspect arcs and remove unsupported values until no further changes are needed.

Basic idea of AC-3

1. Put all arcs in a queue.
2. Remove one arc from the queue.
3. Check whether some values in the first variable's domain have no support in the second variable's domain.
4. If values are removed, then neighboring arcs may be affected, so add them back to the queue.
5. Continue until the queue is empty.

Why rechecking is necessary

If the domain of one variable shrinks, this may destroy support for values in other variables. So the effects must be propagated through the network.

AC-3 Step	Purpose
Initialize queue of arcs	Start with all binary relationships that may need checking
Revise one arc	Remove unsupported values
If a domain changes	Reconsider related arcs because support may be lost elsewhere
Stop when queue is empty	No further domain reduction is possible through arc consistency

13. AC-3 Flow in Map Coloring

Suppose the relevant map-coloring arcs include:

- `(WA, NT)`
- `(NT, WA)`
- `(WA, SA)`
- `(SA, WA)`

Assume:

- `Domain(WA) = {red, green}`
- `Domain(NT) = {red}`
- `Domain(SA) = {red, blue}`

AC-3 may first inspect `(WA, NT)`. Since `WA = red` has no support in `NT`, remove `red` from `WA`. Now:

```
Domain(WA) = {green}
```

Because `WA` changed, arcs involving neighboring variables such as `(SA, WA)` may need rechecking. If `SA` had a value that only worked with the old `WA` domain, that value could now lose support. AC-3 keeps pushing such updates until stability is reached.

14. AC-3 Flow in Exam Scheduling

Suppose the relevant exam-scheduling arcs include:

- `(CS101, CS102)`
- `(CS102, CS101)`
- `(CS101, MATH201)`
- `(MATH201, CS101)`

Assume:

- `Domain(CS101) = {Mon-AM, Tue-AM}`
- `Domain(CS102) = {Mon-AM}`
- `Domain(MATH201) = {Tue-AM}`

AC-3 may inspect `(CS101, CS102)` and remove `Mon-AM` from `CS101`, because it has no support in `CS102`. Then:

```
Domain(CS101) = {Tue-AM}
```

Since `CS101` changed, the arc `(MATH201, CS101)` may need rechecking. If the constraint is `CS101 ≠ MATH201` and `MATH201` only has `Tue-AM`, then that value has now lost support, so `MATH201`'s domain becomes empty. At that point, AC-3 detects failure immediately.

This is a strong example of how propagation can expose a contradiction that plain forward checking might not uncover as early.

> **High-Yield Point:**
>
> AC-3 does not just prune once. It keeps rechecking affected arcs until no more pruning is possible or failure is found.

15. Forward Checking vs Arc Consistency

Feature	Forward Checking	Arc Consistency / AC-3
When applied	Usually after assigning a variable	Can be applied before or during search
Scope	Immediate neighbors of assigned variable	Propagates through arcs across the network
Strength	Weaker	Stronger

Main benefit	Cheap early pruning	Deeper pruning and stronger look-ahead

Forward checking is simpler. Arc consistency is stronger. Both are meant to reduce wasted search.

16. Using Propagation with Backtracking

In practice, propagation is combined with backtracking.

The typical pattern is:

1. Choose a variable and assign a value.
2. Use forward checking or arc consistency to prune domains.
3. If a domain becomes empty, backtrack immediately.
4. If not, continue search with the reduced problem.

This creates a much smarter search process than plain backtracking alone.

> **Big Picture:**
>
> Backtracking explores possibilities. Propagation reduces impossible possibilities before search goes too far.

17. Things to Think About

- Why is it useful to remove impossible values before we explicitly try them?
- Why is forward checking weaker than full propagation?
- Why can AC-3 detect contradictions earlier than plain backtracking?
- Why does local consistency not always guarantee a global solution?

18. Quick Recap

- Forward checking removes inconsistent values from neighboring domains after an assignment.
- Its main purpose is earlier failure detection.
- Constraint propagation is the broader idea of repeatedly shrinking domains using constraints.
- Arc consistency means every value in one domain has supporting value(s) in the neighboring domain.
- AC-3 is a standard algorithm for enforcing arc consistency.
- These methods improve efficiency by pruning impossible branches before search goes too deep.

19. Part 5 Revision Lines

- **Forward checking:** immediate look-ahead after assignment.
- **Constraint propagation:** keep pushing implications of domain reduction.
- **Arc consistency:** every remaining value must have local support.
- **AC-3:** repeated arc revision until no more pruning is possible.

End of Part 5